

MECHATRONICS AND IOT

ASSIGNMENT REPORT – 6

TITLE:

Design and develop an MQTT-Based Remote HVAC Monitoring System using ThingSpeak Cloud. The system remotely monitors HVAC parameters and stores sensor data on cloud platforms for analysis and control.

TEAM MEMBERS:

HARISANKARAN S

(24MER013)

MUTHUKUMAR M

(24MER034)

MONESHWARAN C

(24MER032)

PROJECT OVERVIEW

Heating, Ventilation and Air Conditioning (HVAC) systems are widely used in industries, commercial buildings, hospitals, offices, laboratories, and residential buildings. Continuous monitoring of HVAC parameters is important for maintaining indoor comfort, reducing energy consumption, and preventing equipment failures. The proposed system uses an **ESP32 microcontroller, environmental sensors, Wi-Fi, MQTT communication, and ThingSpeak Cloud** to monitor HVAC conditions remotely. The sensor data is transmitted to the cloud, stored, and displayed through graphs for real-time monitoring and historical analysis.

PROBLEM STATEMENT

- Excessive room temperature.
- High or low humidity.
- Poor indoor air quality.
- Continuous operation of HVAC equipment.
- Increased energy consumption.
- Lack of remote monitoring.
- Delayed detection of abnormal conditions.

PROPOSED SOLUTION

The proposed system uses an **ESP32 controller, DHT22, DS18B20, and MQ-135 sensors** to continuously monitor HVAC parameters. The ESP32 collects and processes the sensor readings according to predefined threshold values. The data is transmitted using **Wi-Fi and MQTT** to a cloud platform, where it can be stored and analyzed. When temperature, humidity, or air quality exceeds the specified limits, the system generates a warning and can activate an alarm.

WORKING PRINCIPLE

The proposed system uses an **ESP32 controller, DHT22, DS18B20, and MQ-135 sensors** to continuously monitor HVAC parameters. The ESP32 collects and processes the sensor readings according to predefined threshold values. The data is transmitted using **Wi-Fi and MQTT** to a cloud platform, where it can be stored and analyzed. When temperature, humidity, or air quality exceeds the specified limits, the system generates a warning and can activate an alarm.

SYSTEM ARCHITECTURE

HVAC Sensors → ESP32 Microcontroller → Wi-Fi Network → MQTT Communication → ThingSpeak Cloud → Cloud Storage & Graphs → Remote Mobile/Web Dashboard

HVAC THRESHOLD VALUES

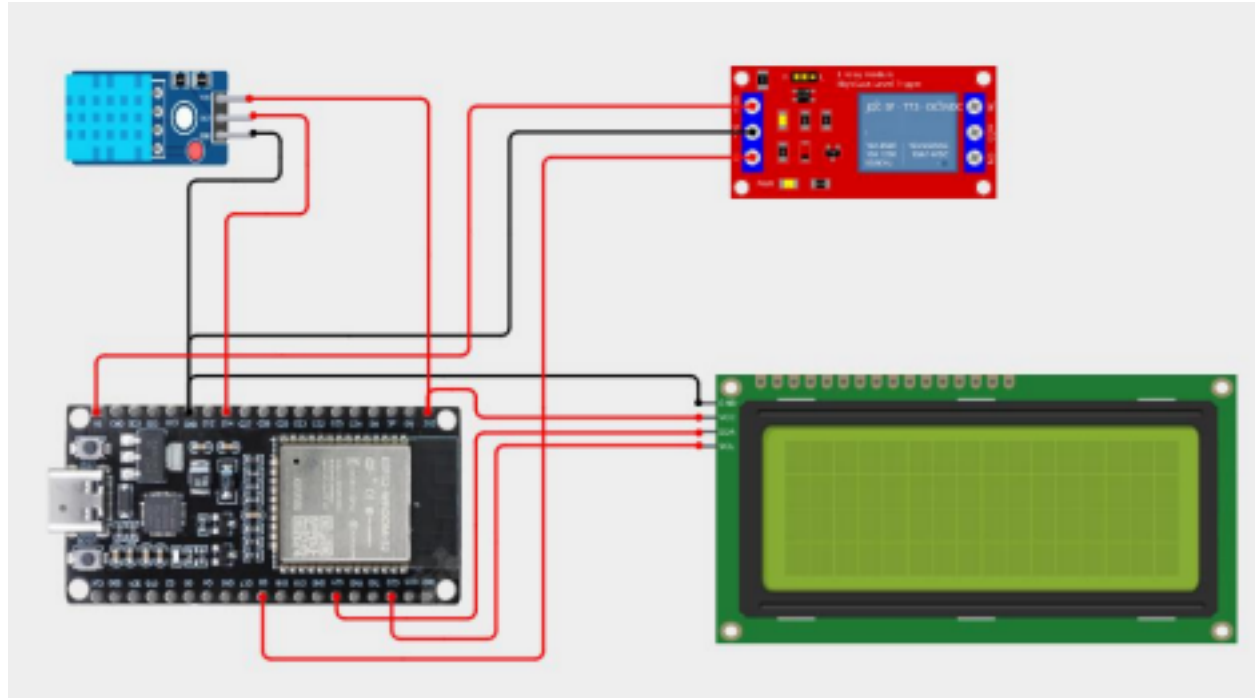
Parameter	Normal Warning Critical		
Room		22–25 °C 25–28 °C >28 °C	

Temperature		40–60 % 60 –70 % >70 %	
Humidity			
Supply-Air Temperature			
18–20 °C >20	<100	100–150 >150	
Temperature 20–25 °C			
°CAir Quality			

PARAMETERS MONITORED

Parameter	Sensor/Device	Purpose
Room		Monitor room
Temperature	DHT22	temperature Monitor
Humidity	DHT22	indoor humidity
Supply-Air Temperature		Monitor supply air
DS18B20 Return-Air		Monitor return air
Temperature DS18B20	MQ-135	Monitor air-quality
Quality	Relay	conditions Monitor fan
Fan Status	Relay	operation
Compressor Status		Monitor compressor op

CIRCUIT DESIGN



ALGORITHM

- **START**
- Initialize the ESP32 microcontroller.
- Initialize the DHT22, DS18B20, and MQ-135 sensors.
- Initialize the LCD and buzzer.
- Connect the ESP32 to the Wi-Fi network.
- Connect to the MQTT broker.
- Read room temperature and humidity.
- Read supply-air and return-air temperatures.
- Read the air-quality value.
- Monitor fan and compressor status.
- Compare all readings with predefined HVAC threshold values.
- Determine the HVAC condition as **NORMAL**, **WARNING**, or **CRITICAL**. •
- Display the readings on the LCD.
- Publish the sensor data using MQTT.
- Upload/store the data on ThingSpeak Cloud.
- Activate the buzzer when a critical condition is detected. •

Repeat the process continuously.

• **STOP**

CODING

```
//  
=====
```

// REMOTE HVAC MONITORING SYSTEM
// ESP32 + DHT11 + OLED + RELAY + ADAFRUIT IO
// =====

```
#include <WiFi.h>  
#include "AdafruitIO_WiFi.h"  
#include <DHT.h>  
#include <Wire.h>  
#include <Adafruit_GFX.h>  
#include <Adafruit_SSD1306.h>
```

```
//  
=====
```

// WIFI DETAILS
// =====

```
#define WIFI_SSID "OnePlus Nord CE5 7C5C"  
#define WIFI_PASS "qkts5737"
```

```
//  
=====
```

// ADAFRUIT IO DETAILS
// =====

```
// Use the SAME Adafruit IO username and IO key  
// that you used in your previous RFID project.
```

```
#define IO_USERNAME "Hair_13"  
#define IO_KEY "aio_EMRC40vci7DQ5Amz5kvOKh6WKHI0"
```

```
//  
=====
```

// ADAFRUIT IO CONNECTION
// =====

```
AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);  
  
//
```

```
=====
// ADAFRUIT IO FEEDS
```

```
//
```

```
=====
AdafruitIO_Feed *temperatureFeed =
```

```
  io.feed("temperature");
```

```
AdafruitIO_Feed *humidityFeed =
```

```
  io.feed("humidity");
```

```
AdafruitIO_Feed *hvacStatusFeed =
```

```
  io.feed("hvac-status");
```

```
//
```

```
=====
// ESP32 PINS
```

```
// =====
```

```
// DHT11
```

```
#define DHT_PIN 4
```

```
#define DHT_TYPE DHT11
```

```
// Relay
```

```
#define RELAY_PIN 5
```

```
// OLED
```

```
#define OLED_SDA 21
```

```
#define OLED_SCL 22
```

```
//
```

```
=====
// DHT SENSOR
```

```
//
```

```
=====
DHT dht(DHT_PIN, DHT_TYPE);
```

```
//
```

```
=====
// OLED
```

```
// =====
```

```

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(
  SCREEN_WIDTH,
  SCREEN_HEIGHT,
  &Wire,
  -1
);

//
=====
// VARIABLES
// =====

float temperature = 0;
float humidity = 0;

bool hvacStatus = false;

unsigned long lastUpdate = 0;

const unsigned long updateInterval = 15000;

//
=====
// FUNCTION DECLARATIONS
// =====

void readSensor();
void displayData();
void controlHVAC();

//
=====
// SETUP
// =====

void setup()
{
  Serial.begin(9600);

  delay(1000);

  //
=====

```

```

// RELAY
//

=====

pinMode(RELAY_PIN, OUTPUT);

digitalWrite(RELAY_PIN, LOW);

//

=====

// DHT11
//

=====

dht.begin();

//

=====

// OLED
// =====
Wire.begin(OLED_SDA, OLED_SCL);

if (!display.begin(
    SSD1306_SWITCHCAPVCC,
    0x3C))
{
    Serial.println("ERROR: OLED NOT DETECTED!");

    while (1);
}

display.clearDisplay();

display.setTextSize(1);
display.setTextColor(SSD1306_WHITE);

display.setCursor(0, 20);
display.println("HVAC MONITOR");

display.setCursor(0, 35);
display.println("Starting...");

display.display();

```

```

delay(2000);

//
=====

// SYSTEM INFORMATION
// =====

Serial.println();
Serial.println("=====")
; Serial.println(" REMOTE HVAC MONITORING SYSTEM");
Serial.println("=====")
; Serial.println("ESP32 + DHT11 + OLED + RELAY");
Serial.println("Adafruit IO Cloud");
Serial.println("=====");

//
=====

// CONNECT TO ADAFRUIT IO
// =====

Serial.println();
Serial.println("Connecting to Adafruit IO...");

io.connect();
while (io.status() < AIO_CONNECTED)
{
  Serial.print(".");
  delay(500);
}

Serial.println();
Serial.println("Adafruit IO Connected!");

//
=====

// INITIAL CLOUD STATUS
//

=====

hvacStatusFeed->save("OFF");

Serial.println();
Serial.println("HVAC Monitoring Ready");
Serial.println();

```

```

}

//
=====

// LOOP
// =====

void loop()
{
  //
  =====
  // KEEP ADAFRUIT IO CONNECTION ALIVE
  //
  =====

  io.run();

  //
  =====
  // UPDATE EVERY 15 SECONDS
  // =====

  if (millis() - lastUpdate >= updateInterval)
  {
    lastUpdate = millis();

    // =====
    // READ DHT11
    // =====

    readSensor();

    // =====
    // CONTROL HVAC
    // =====

    controlHVAC();

    // =====
    // DISPLAY ON OLED
    // =====

    displayData();
  }
}

```

```

// =====
// SEND DATA TO ADAFRUIT IO
// =====

temperatureFeed->save(temperature);

humidityFeed->save(humidity);

if (hvacStatus == true)
{
    hvacStatusFeed->save("ON");
}
else
{
    hvacStatusFeed->save("OFF");
}

Serial.println("Data sent to Adafruit IO");
Serial.println("-----");
}
}

//
=====
// READ DHT11
// =====

void readSensor()
{
    humidity = dht.readHumidity();

    temperature = dht.readTemperature();

    // =====
    // SENSOR ERROR
    // =====

    if (isnan(humidity) || isnan(temperature))
    {
        Serial.println("ERROR: DHT11 reading failed!");

        return;
    }

    //
=====

```

```

// SERIAL MONITOR
// =====

Serial.println();
Serial.println("HVAC SENSOR DATA");

Serial.print("Temperature: ");
Serial.print(temperature);
Serial.println(" °C");

Serial.print("Humidity: ");
Serial.print(humidity);
Serial.println(" %");
}

//
=====
// HVAC CONTROL
// =====

void controlHVAC()
{
//
=====
// SIMPLE TEMPERATURE CONTROL
//
=====
//
// If temperature is above 28°C:
// HVAC = ON
//
// If temperature is 28°C or below:
// HVAC = OFF
//
// =====

if (temperature > 28)
{
digitalWrite(RELAY_PIN, HIGH);

hvacStatus = true;

Serial.println("HVAC STATUS: ON");
}
else
{

```

```

    digitalWrite(RELAY_PIN, LOW);

    hvacStatus = false;

    Serial.println("HVAC STATUS: OFF");
}
}

//
=====
// OLED DISPLAY
// =====

void displayData()
{
    display.clearDisplay();

    display.setTextSize(1);
    display.setTextColor(SSD1306_WHITE);

    // Title
    display.setCursor(0, 0);
    display.println("HVAC MONITOR");

    // Temperature
    display.setCursor(0, 18);

    display.print("Temp: ");

    display.print(temperature, 1);

    display.println(" C");

    // Humidity
    display.setCursor(0, 32);

    display.print("Humidity: ");

    display.print(humidity, 1);
    display.println(" %");

    // HVAC
    display.setCursor(0, 48);

    display.print("HVAC: ");

```

```

if (hvacStatus == true)
{
  display.println("ON");
}
else
{
  display.println("OFF");
}

display.display();
}

```

SYSTEM WORKING

The proposed MQTT-Based Remote HVAC Monitoring System uses an **ESP32 microcontroller** to continuously monitor important HVAC parameters. The **DHT22** measures room temperature and humidity, while two **DS18B20 sensors** measure supply-air and return-air temperatures. The **MQ-135** monitors air-quality conditions. The ESP32 collects the sensor readings and compares them with predefined HVAC threshold values. The current readings are displayed on the LCD. The ESP32 then sends the sensor data through **Wi-Fi using MQTT communication** to the cloud platform. ThingSpeak stores the data and provides graphs for real-time monitoring and historical analysis. If any parameter exceeds the specified limit, the system identifies the condition as **WARNING or CRITICAL** and activates the buzzer during critical conditions. The monitoring process continues continuously.

COMPONENTS

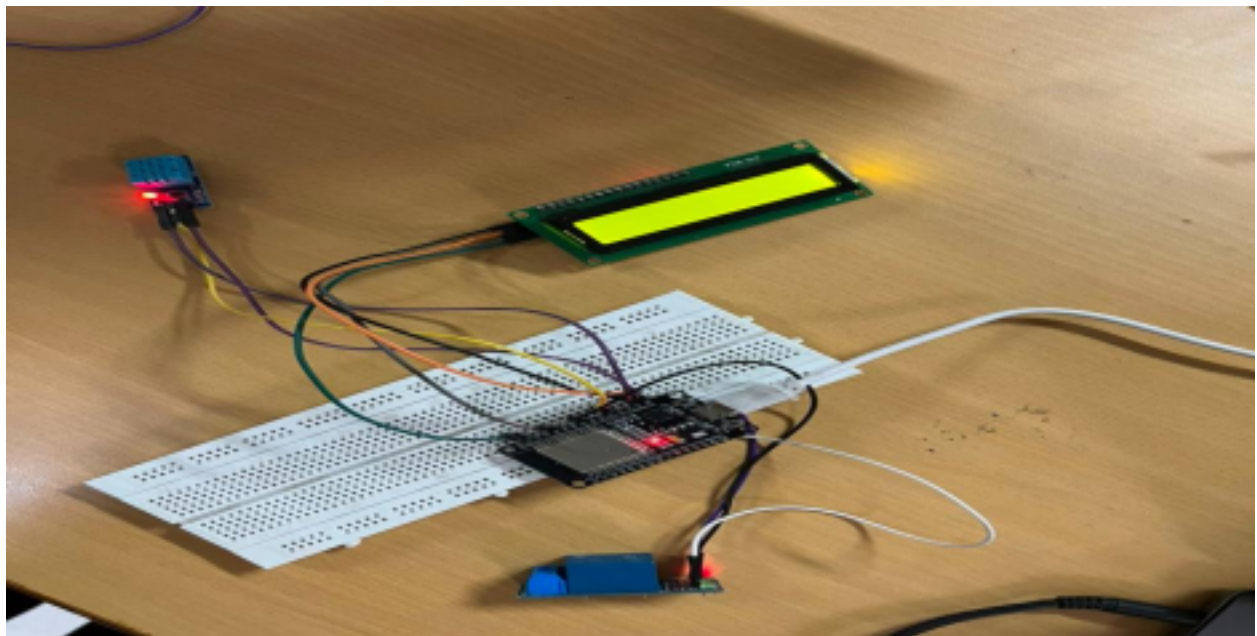
S. No	Component	Quantity	Purpose
1			Main controller
2			Room temperature & humidity
3	ESP32 Dev Board	1	Supply and return air temperature
4	DHT22 Sensor	1	Air-quality monitoring
5	DS18B20 Sensor	2	Fan and compressor control/status
6			Display HVAC parameters
7			Warning indication
8			

9	MQ-135 Air Quality Sensor	1	Circuit connections Power source
	Relay Module	2	
	16×2 I2C LCD	1	
	Buzzer & LEDs	1 Set	
	Breadboard & Jumper Wires	1 Set	
	Power Supply	1	

PROTOTYPE IMPLEMENTATION

The prototype is designed around an **ESP32 controller and multiple HVAC sensors**. The DHT22, DS18B20, and MQ-135 sensors are connected to the specified ESP32 pins for collecting environmental and air-quality data. The LCD provides real-time display of HVAC parameters, while the buzzer provides an alert during critical conditions.

The ESP32 communicates through Wi-Fi and MQTT. The monitored HVAC data is transmitted to the cloud platform for storage and visualization. The prototype therefore provides both **local monitoring and remote cloud-based monitoring**.



RESULTS & PERFORMANCE ANALYSIS

The developed system provides real-time monitoring of HVAC environmental parameters. The sensor readings are compared with predefined threshold values to identify normal, warning, and critical operating conditions.

Parameter		Normal	Warning	Critical
Room Temperature		22–25 °C	25–28 °C	>28 °C
Humidity		40–60 %	60–70 %	>70 %
Supply-Air Temperature				
18–20 °C				
>20 °C				
Air Quality	<100	100–150		>150

PERFORMANCE

- Provides **real-time HVAC parameter monitoring**.
- Uses **MQTT-based wireless communication**.
- Provides **remote monitoring through ThingSpeak Cloud**.
- Automatically detects abnormal temperature, humidity, and air-quality conditions.
- Provides a **local buzzer alert** during critical conditions.
- Stores historical sensor data for HVAC performance analysis.
- Provides graphical visualization of monitored data.
- Helps identify abnormal HVAC operating conditions quickly.
- Provides a **low-cost prototype** suitable for educational and small-scale IoT applications.

